

Chapter 2. Software Testing Fundamentals

- 2.1. Definition & Objectives of Testing
- 2.2. Role of testing and its effect on quality
- 2.3. Causes of software failure: Definition of -Error, Bug, Fault, Defect and Failure
- 2.4. Economics of Testing
- 2.5. Seven Testing Principles
- 2.6. Software Testing Life cycle
- 2.7. Validation & Verification Concepts - V Model and W Model
- 2.8. Agile Testing- Test Driven Software Development
- 2.9. Levels of Testing-
 - 2.9.1. Unit (Component) Testing
 - 2.9.2. Integration Testing
 - 2.9.3. System Testing
 - 2.9.4. User Acceptance Testing (UAT)
- 2.10. Test Types
 - 2.10.1. Functional testing (Black-box)
 - 2.10.2. Non-functional testing (Testing of software product characteristics)
 - 2.10.3. Structural testing (White-box)
 - 2.10.4. Testing related to changes – Confirmation (Re-testing) and Regression Testing
- 2.11. Non-Functional Testing Types –
 - 2.11.1. Performance (Load & Stress)
 - 2.11.2. Usability
 - 2.11.3. Maintainability
 - 2.11.4. Portability
 - 2.11.5. Security
 - 2.11.6. Localization & Internationalization
- 2.12. Concept of Smoke testing and Sanity Testing

2.1. Definition & Objectives of Testing

Definition of Testing

Software testing is the process of evaluating a software application to identify defects, ensure it meets the specified requirements, and verify that it performs as expected.

In simpler terms, testing is checking if a system or application works correctly and does what it is supposed to do.

Objectives of Testing

- **Find and Fix Defects (Bugs):** The primary goal of testing is to identify errors or bugs in the software so that they can be fixed before the product is released to users.
Example: If a mobile banking app crashes when transferring money, testing helps find this issue and fix it.
- **Ensure Quality:** Testing ensures that the software meets high-quality standards, including functionality, performance, and user experience.
Example: Ensuring an e-commerce website loads quickly and functions well on all devices.
- **Verify Requirements:** Testing checks whether the software meets the requirements and expectations defined at the start of the project.
Example: If the requirement is that users should be able to upload photos, testing verifies this functionality works correctly.
- **Prevent Future Issues:** By identifying bugs early in development, testing helps prevent them from causing major problems later.
Example: Catching a small glitch in a feature during testing prevents it from becoming a critical issue in production.
- **Improve User Experience:** Testing ensures that the software is easy to use and meets the needs of end users.
Example: Testing ensures a navigation menu is intuitive and accessible.
- **Validate Functionality:** Ensures all features of the software work as expected, both individually and together.
Example: Testing a login page to verify the correct username and password allow access.
- **Ensure Security:** Testing identifies vulnerabilities in the software to protect it from hacking and unauthorized access.
Example: Ensuring strong password requirements and preventing unauthorized data access.

- **Enhance Reliability and Stability:** Ensures the software performs reliably under various conditions and doesn't fail unexpectedly.
Example: Testing an airline booking system to handle thousands of simultaneous users without crashing.
- **Reduce Costs:** Identifying and fixing defects early in development is cheaper than dealing with problems after deployment.
Example: Fixing a bug during testing costs less than addressing customer complaints post-release.

2.2. Role of testing and its effect on quality

Role of Testing in Software Development

1. Identifying Defects:

- Testing plays a crucial role in finding bugs or defects in the software before it reaches the users.
- Example: Testing a shopping app to ensure the "Add to Cart" feature works correctly.

2. Ensuring Software Reliability: It ensures the system behaves consistently and performs well under different conditions.

3. Validating Requirements:

- Testing checks if the software meets the specified requirements and works as expected.
- Example: Verifying that a ticket-booking app allows seat selection as per the client's requirements.

4. Enhancing User Experience: By detecting and fixing issues, testing ensures the application is easy to use and provides a smooth user experience.

5. Maintaining Security: Testing protects sensitive data and prevents unauthorized access by identifying vulnerabilities.

6. Reducing Development Costs:

- Catching defects early in the development cycle reduces the cost and effort of fixing them later.
- Example: Fixing a bug during the coding phase is cheaper than addressing it after

deployment.

7. Ensuring Compatibility:

- Testing ensures the software runs smoothly on various platforms, devices, or browsers.
- Example: Testing a website to work seamlessly on Chrome, Safari, and Firefox.

Effects of Testing on Quality

1. **Improved Product Quality:** Regular testing ensures the software meets the required quality standards and performs well under all scenarios
2. **Increased User Satisfaction:** A well-tested application provides a smooth and error-free experience, leading to satisfied customers.
3. **Higher Reliability:** In-depth testing ensures the system functions consistently and reliably.
4. **Reduced Risk of Failure:** Testing minimizes the chances of system failure in production environments.
5. **Enhanced Security:** Security testing detects and eliminates vulnerabilities, protecting users' data and privacy.
6. **Lower Maintenance Effort:** A well-tested product has fewer bugs in production, reducing maintenance costs and effort.

2.3. Causes of software failure: Definition of -Error, Bug, Fault, Defect and Failure

- **Error** : An **error** is a mistake made by a person (like a developer) when writing the code. It's like a human slip-up. A human mistake made by a developer, designer, or tester while writing code or designing the system.

Real-Life Example:

- A student calculates $2+2=5$ in math exam → This is an error
- A cook puts salt instead of sugar in cake → This is an error

Example: A developer accidentally types the wrong variable name.

Example

```
// Programmer wanted to add two numbers
```

```
int a = 5, b = 3;
```

```
int sum = a - b; // ERROR! Programmer typed '-' instead of '+'
```

```
// He meant to add but subtracted by mistake
```

- **Bug** : A **bug** is a problem in the software caused by an error in the code. It makes the software behave in a way that wasn't expected. A bug is the actual wrong code sitting in the program.

Example: The app crashes when you try to upload an image.

- **Fault** : A **fault** is like a hidden problem in the code. It's something wrong that might not cause an issue right away, but it can cause problems later.

Example: A piece of code that divides by zero, which doesn't cause a problem unless it's actually used.

Interesting Fact: Many engineers use Bug and Fault interchangeably, but technically:

Bug = Wrong code written

Fault = Wrong result when that code executes

// Same example

```
int calculateSum(int a, int b) {
```

```
    return a - b; // This is the BUG
```

```
}
```

// When program RUNS:

// User enters 5 and 3

// Program executes the BUG

// It shows 2 instead of 8

// This WRONG OUTPUT is the FAULT

Analogy:

- **Error:** Cook decides to put salt instead of sugar (human mistake)
- **Bug:** Recipe written as "add salt" instead of "add sugar" (wrong instruction)
- **Fault:** Cake tastes salty (wrong result when recipe followed)

- **Defect:** Defect is Mismatch between expected result and Actual result. Error found by the tester is called **Defect**. Defect is when software does NOT do what the customer wants.

Example: If there's a button to reset your password but it doesn't do anything when you click it, that's a defect.

- **Failure:** A **failure** is when the software completely stops doing its job. This happens when a defect (or problem) in the code causes the system to fail. A failure happens when the end user experiences the problem while using the software. It's when the software crashes, stops working, or gives wrong results to the user.

Example: A website crashes completely, and users can't even open the homepage.

COMPARISON TABLE

Basis	ERROR	BUG	FAULT	DEFECT	FAILURE
Definition	Human mistake made by developer	Presence of wrong code in program	Incorrect step/process in software	Deviation from customer requirement	Software malfunctions during execution
Nature	Action (what human did wrong)	Static (wrong code sitting in program)	Dynamic (wrong behavior when code runs)	Requirement gap (not what customer wanted)	User-visible problem
When it occurs	During development/coding	After coding, before execution	During program execution	During testing vs requirements	During actual use by end-user
Who introduces it	Programmer/Developer	Programmer/Developer	System/Program itself	Miscommunication between customer & developer	System failure during operation
Can it be detected?	Yes, through code reviews	Yes, through testing	Yes, during execution	Yes, through requirement validation	Yes, when user uses software
Impact	Potential to create bugs	Potential to cause faults	Causes incorrect behavior	Makes software unfit for purpose	User dissatisfaction, loss of data

Example	Programmer typed = instead of ==	Code has if(x = 5) instead of if(x == 5)	Program always enters wrong condition	App shows price without tax (customer wanted with tax)	App crashes, shows wrong calculation
----------------	-------------------------------------	--	---	---	---

2.4. Economics of Testing

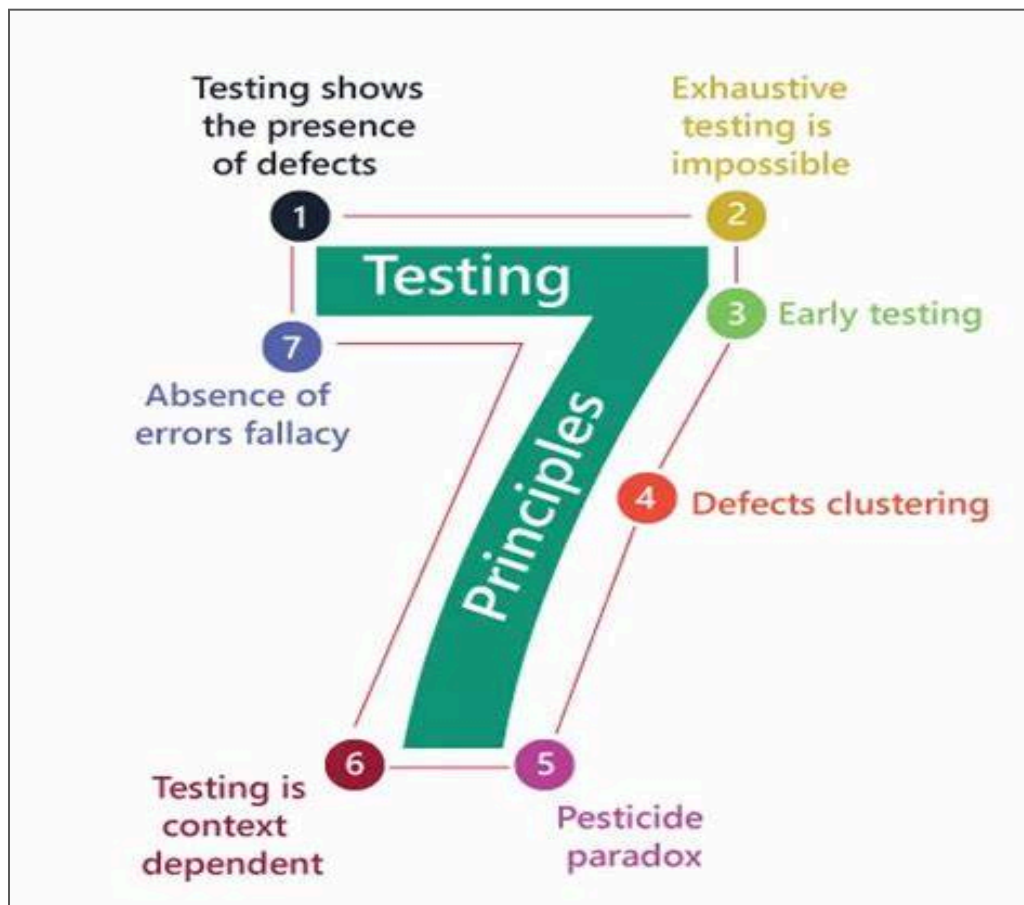
The **economics of testing** refers to the balance between the costs involved in testing software and the benefits it brings. It focuses on understanding the financial impact of testing activities, helping organizations make decisions about the amount of testing to perform based on available resources and the risks involved.

The goal is to figure out how much testing is needed without wasting time and money.

1. **Cost of Testing:** Testing takes time, tools, and people. It can get expensive, especially if the software is large or complex. It's important to plan well so you don't spend too much on testing.
2. **Benefits of Testing:** Testing helps to find bugs early before the software is released. This saves money and avoids problems later. It also helps improve the software quality, making users happy and preventing issues that could hurt the company's reputation.
3. **Cost of Fixing Bugs:** Bugs are cheaper to fix when they are found early in development. The later you find them (like after the software is released), the more expensive and harder they are to fix.
4. **Return on Investment (ROI):** Good testing gives a high return on investment. By finding bugs early, you save money and avoid expensive problems after release. Testing can be expensive upfront, but it prevents bigger costs later.
5. **Risk-Based Testing:** In **risk-based testing**, you focus more on testing the parts of the software that are most likely to have problems or cause big issues. This helps save time and resources by testing the most important parts carefully.

To sum it up, the economics of testing is about doing enough testing to make sure the software works well without spending too much money or time.

2.5. Seven Testing Principles



1. Testing shows the presence of defects

Testing can't prove that the software is completely bug-free. It can only show that there *are* defects. So, the goal of testing is to find problems, not to guarantee perfection.

Example: Just because a test passes, doesn't mean there are no hidden bugs.

2. Exhaustive testing is impossible

It is the process of testing the functionality of the software in all possible inputs (valid or invalid) and pre-conditions is known as exhaustive testing. You can't test every possible scenario in a program because there are just too many possibilities. Instead, testers focus on the most important or risky parts of the software. Exhaustive testing is impossible means the software can never test at every test case.

Example: It's impossible to test every combination of buttons a user might press, so you focus on the main features.

3. Early testing

Testing should start as early as possible in the development process. Finding and fixing bugs early is cheaper and easier than finding them later.

Example: Testing the first version of the code helps catch bugs before the program grows more complex.

4. Defects cluster together / Defect Clustering

This principle means that if one part of the software has a bug, there are likely to be more bugs in the same area. It's like if one part of a machine breaks down, other parts nearby might also be broken or need fixing.

Example: If there's a bug in the login feature, you might also find other bugs in related areas like the password reset or user authentication. So, instead of testing random parts of the software, testers focus on the parts that are most likely to have multiple issues.

5. Beware of the pesticide paradox

Running the same tests over and over won't help find new bugs. You need to change your test cases or create new ones to find different defects.

Example: If you always run the same test for a feature, you may not discover new bugs. You need to try different test scenarios.

6. Testing is context-dependent

The way you test software depends on the type of software, its purpose, and how critical it is. Testing for a mobile app is different from testing for a banking system.

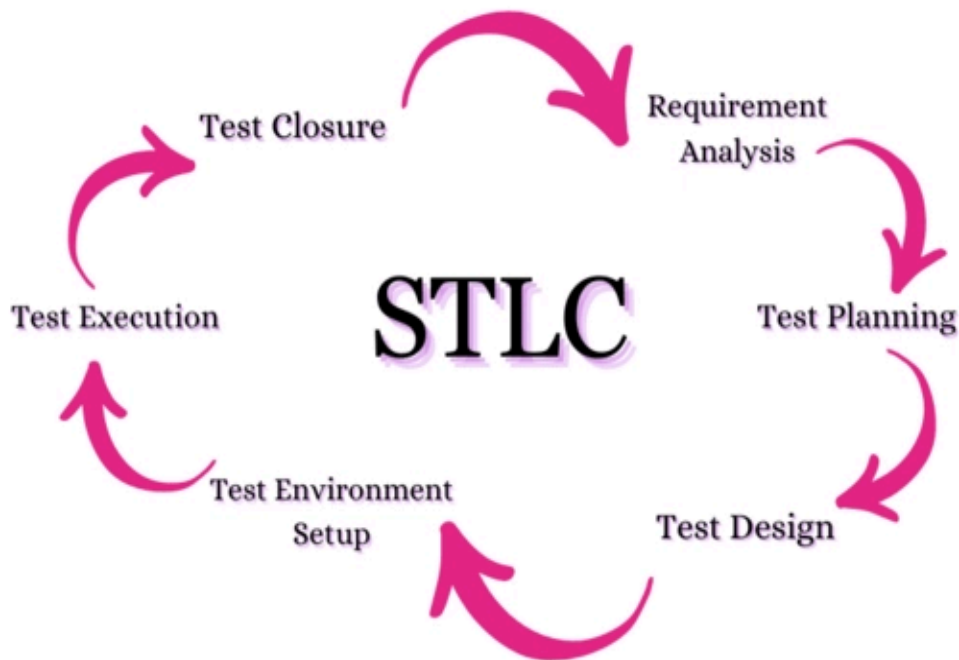
Example: A simple mobile app may only need basic testing, but a banking app needs thorough security testing to protect users' data.

7. Absence of errors is a fallacy

If a built software is 99% bug-free but does not follow the user requirement then it is unusable. It is not only necessary that software is 99% bug-free but it is also mandatory to fulfill all the customer requirements.

Example: The app may run perfectly without crashes, but if it doesn't give users the features they expect, it's still a failure.

2.6. Software Testing Life cycle



1. Requirement Analysis :

Requirement Analysis is the first step of the Software Testing Life Cycle (STLC). In this phase the quality assurance team understands the requirements like what is to be tested. If anything is missing or not understandable then the quality assurance team meets with the stakeholders to better understand the detailed knowledge of requirements.

Example: A company is developing an **e-commerce website**. The testing team analyzes requirements such as user login, product search, and payment gateway to identify testable scenarios.

2. Test Planning:

The test strategy is prepared, and the scope of testing is defined. The required tools, resources, and timelines are decided. This phase gets started once the requirement-gathering phase is completed.

Example: The test manager creates a plan that includes the number of testers, test environment setup, manual or automation testing, and estimated testing time.

3. Test Case Development (Test Design):

The test case development phase gets started once the test planning phase is completed. In this phase the testing team notes down the detailed test cases. The testing team also prepares the

required test data for the testing. When the test cases are prepared then they are reviewed by the quality assurance team.

4. Test Environment Setup :

Setup of the test environment is an independent activity and can be started along with Test Case Development.

Test Environment Setup is the process of preparing the hardware, software, network, and test data required for executing test cases. This phase includes configuring servers, databases, test tools, and necessary system dependencies

Proper environment setup helps in detecting and resolving issues early, ensuring smooth test execution.

In this process, the testing team is not involved. either the developer or the customer creates the testing environment.

5. Test Execution:

After the test case development and test environment setup, the test execution phase gets started. In this phase the testing team starts executing test cases based on prepared test cases in the earlier step. The test cases are executed, and any defects found are reported to the development team.

6. Test Closure

Once testing is completed, a final report is prepared, summarizing the testing process, defects found, and lessons learned.

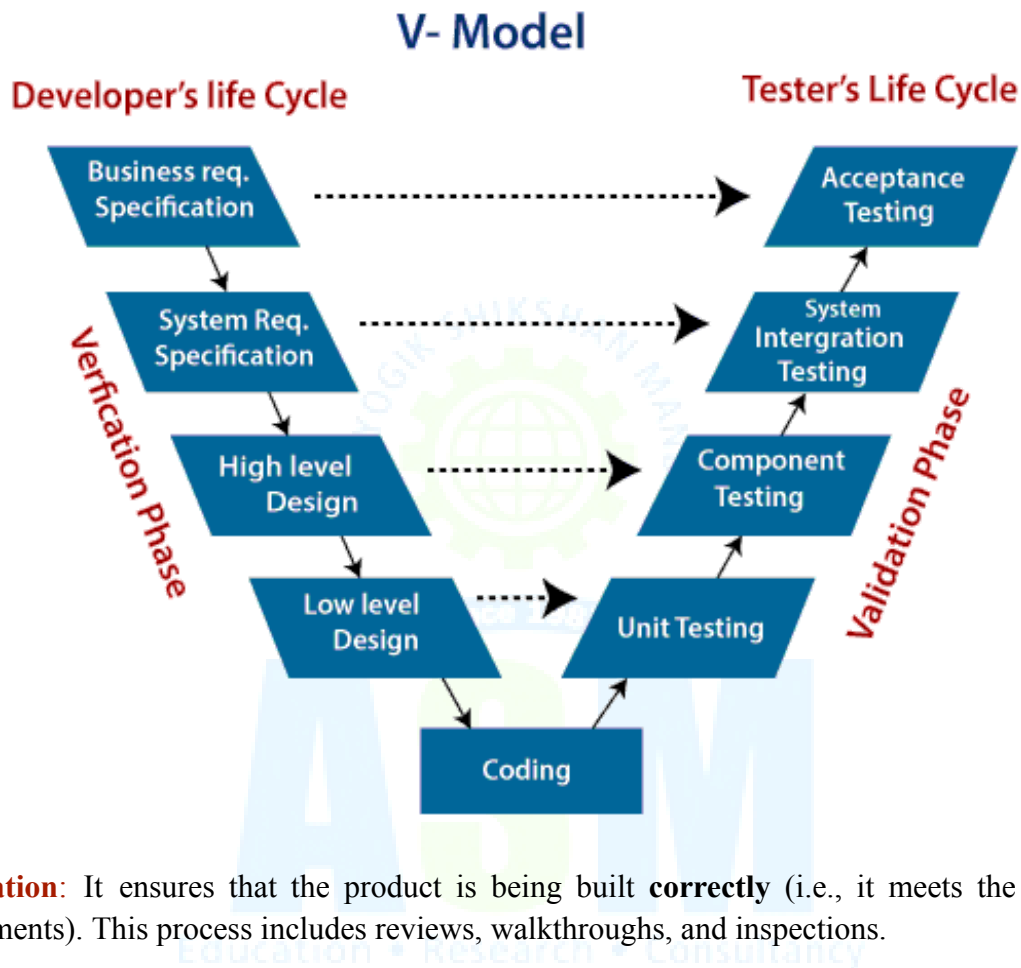
Example: The test team creates a **Test Summary Report** showing:

- Total test cases executed
- Number of defects found and fixed
- Overall software quality assessment

After final approval, the e-commerce website is ready for release.

2.7. Validation & Verification Concepts - V Model and W Model

V-Shaped Structure: The process is visually represented in the shape of the letter "V". The left side of the "V" focuses on development activities (such as requirement analysis, design, coding), and the right side represents testing activities (such as unit testing, integration testing, and system testing).



Verification: It ensures that the product is being built **correctly** (i.e., it meets the specified requirements). This process includes reviews, walkthroughs, and inspections.

1. Business Requirement Specification

- The project team gathers business requirements from stakeholders.
- These requirements define what the system should do.
- **Corresponding Testing: Acceptance Testing** (Ensures the final product meets business needs).

2. System Requirement Specification

- Detailed system requirements are defined based on business needs.
- Includes functional and non-functional requirements.
- **Corresponding Testing: System Integration Testing** (Verifies that system components work together).

3. High-Level Design (HLD)/Architectural Design

- Defines system architecture, modules, and overall structure.
- Focuses on **how** the system will function.
- **Corresponding Testing: Component Testing** (Ensures that modules are correctly integrated).

4. Low-Level Design (LLD)/Module Design

- Detailed design of individual components, classes, and functions.
- Defines data structures, algorithms, and internal logic.
- **Corresponding Testing: Unit Testing** (Verifies individual components work as expected).

5. Coding

- The actual implementation of software modules takes place based on LLD.
- The **output** of this phase is the source code of the application.
- Next, the validation phase starts to test the code.

Validation (Testing Side): It ensures that the **right product** is being built (i.e., it meets the user's needs and expectations). This includes testing and user acceptance activities.

1. Unit Testing (Low-Level Testing)

- Tests individual units (functions, methods, classes).
- Performed using tools like JUnit, NUnit, and PyTest.

2. Component Testing (Integration Testing)

- Tests how different components interact with each other.
- Ensures that data flow between modules works correctly.

3. System Integration Testing (SIT)

- Verifies that the entire system works as a whole.
- Focuses on interfaces, APIs, and external system interactions.

4. Acceptance Testing

- Ensures that the final system meets business requirements.
- Usually performed by the **end-users** before deployment.

W Model

Paul Herzlich introduced the W-Model in 1993. W-model is the most recent software development model where we start real testing activity simultaneously software development process

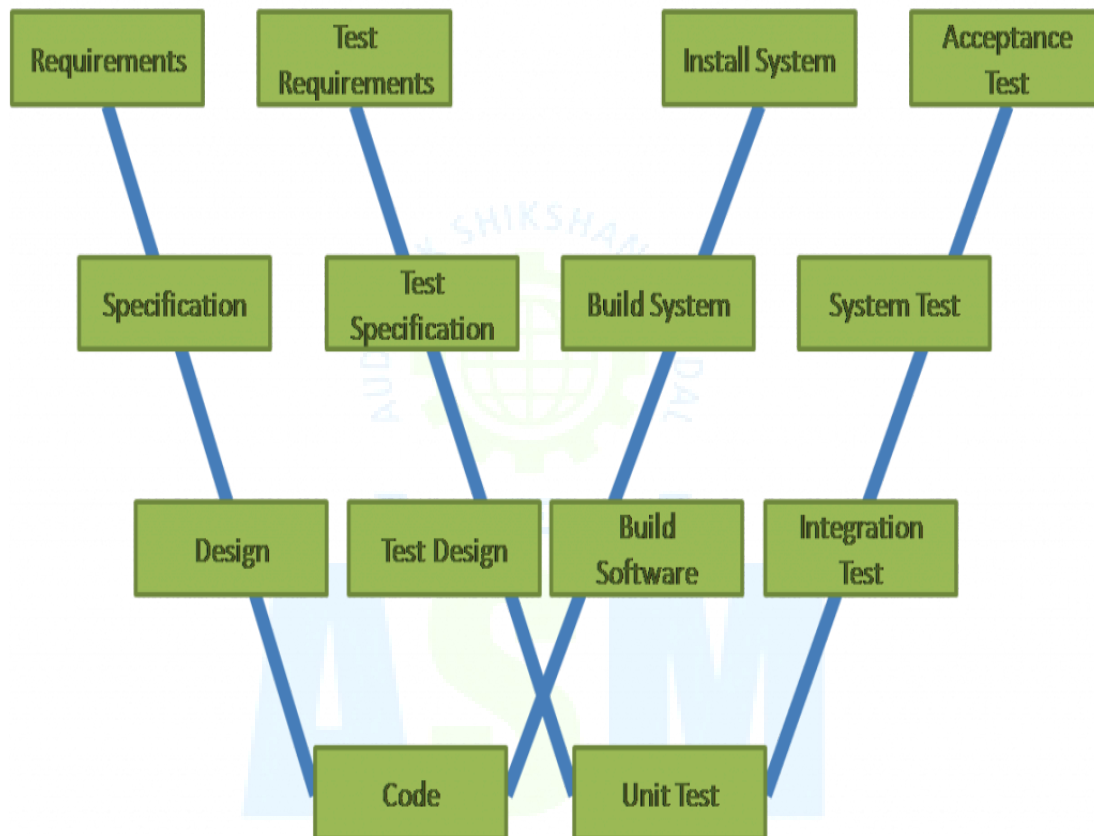
The W-Model is an improved version of the V-Model that focuses on testing at every stage of

development. It ensures that testing is not just done at the end but runs parallel to every phase of software development.

It is called the **W-Model** because it looks like the letter **W**.

It has **two V's**:

- **First V** for **Development stages**.
- **Second V** for **Testing stages**.



Development Phases

Requirements:

- This is where the needs of the project are collected from the client or user.
- Example: "The app should have a login feature, shopping cart, and payment options."

Specification:

- Write down the detailed requirements and functionalities that the system must have.
- Example: "The login feature will include email and password authentication."

Design:

- Plan how the system will be built, including architecture, modules, and data flow.
- Example: Design the layout of the home page, product page, and cart page.

Code:

- Write the actual code for each part of the system according to the design.
- Example: Write code for the login page, product listing, and checkout process.

Build Software:

- Combine all the code and build the system.
- Example: Integrate the login, product, cart, and payment modules into one system.

Build System:

- Install the software system and prepare it for testing.
- Example: Deploy the app to a testing server for further tests.

Install System:

- Install the final system for acceptance testing.
- Example: Install the final app version for user testing.

Testing Phases

Test Requirements:

- Plan how the software will be tested to ensure it meets all requirements.
- Example: Plan tests to check if users can login successfully or add items to the cart.

Test Specification:

- Write detailed instructions on how each part will be tested.
- Example: "Verify that the login page accepts correct credentials and rejects wrong ones."

Test Design:

- Design specific test cases for each module.
- Example: Design tests for the "Add to Cart" button to check if items are added correctly.

Unit Test:

- Test each piece of code after it is written.
- Example: Test if the "Buy Now" button works as expected after coding it.

Integration Test:

- Test how different parts of the system work together.
- Example: Check if the app correctly adds items to the cart and processes payments without errors.

System Test:

- Test the entire system to ensure all features work correctly.
- Example: Verify that users can search for products, add them to the cart, and make payments smoothly.

Acceptance Test:

- Final test to ensure the software meets all client and user needs.
- Example: The client checks if the app is user-friendly and meets all requirements.

Advantages of W-Model in Software Testing:

- **Testing at every stage:** Reduces the chance of big mistakes.
- **Find errors early:** Saves time and cost.
- **High-quality software:** Ensures every part works well.

Example of W-Model in Real Life:

- Think of building a house:
 - **Plan the design** and check if it meets the owner's needs.
 - **Build the walls** and test if they are strong.
 - **Add electricity** and check if all wires work.
 - **Finally, check the whole house** before handing it over.

2.8. Agile Testing- Test Driven Software Development

Agile Testing:

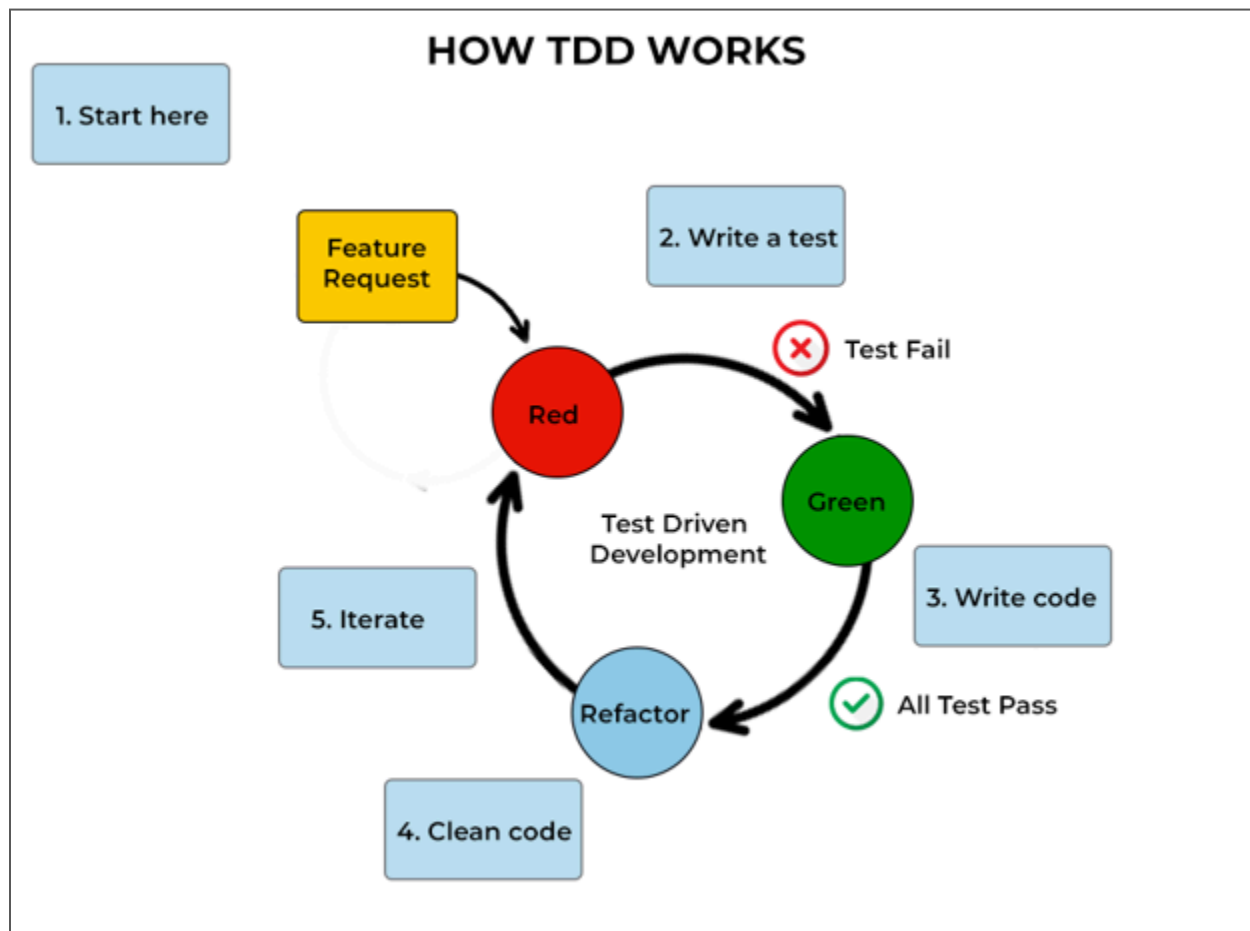
- Agile Testing is a testing process that follows the Agile methodology.
- In Agile, the software is developed in small steps called iterations or sprints (usually 1-2 weeks long).
- Testing happens continuously during each sprint, not at the end of the project.
- Testers, developers, and customers work together in Agile, making sure the product meets the customer's needs quickly.

Key Features of Agile Testing:

- Fast and continuous testing.
- Early bug detection since testing is done after every small change.
- Customer feedback is taken after every sprint.
- Flexible and adaptable to changes in requirements.

Agile Testing Process:

- **User stories** are created (small features or functionalities that the software should have).
- Testers write **test cases** based on these user stories.
- After development, each part is tested immediately.
- Any defects found are fixed quickly, and testing is repeated.



What is Test-Driven Development (TDD)?

- **Test-Driven Development** is a technique in Agile where **tests are written before writing the actual code**.
- It follows a cycle called the **Red-Green-Refactor** cycle:
 1. **Red** – Write a test that fails initially (because there's no code yet).
 2. **Green** – Write just enough code to make the test pass.
 3. **Refactor** – Improve the code while making sure the test still passes.

Steps in Test-Driven Development (TDD):

- **Add a Test:** Write a simple test for the feature you want to implement.
- **Run the Test:** Since the code isn't written yet, the test will fail (this is expected).
- **Write Code:** Write the minimum code required to make the test pass.
- **Run All Tests:** Run all tests to ensure the new code doesn't break any existing features.
- **Refactor Code:** Clean up the code while ensuring all tests still pass.
- **Repeat:** Continue this process for every new feature or change.

Example of Test-Driven Development:

Imagine you are developing a **login system** for an application.

Step 1: Write a test: *"Check if the login works with a correct username and password."*

Step 2: Run the test – it fails because there is no code yet.

Step 3: Write code for the login function.

Step 4: Run the test again – now it passes because the code works.

Step 5: Refactor the code to make it cleaner and more efficient.

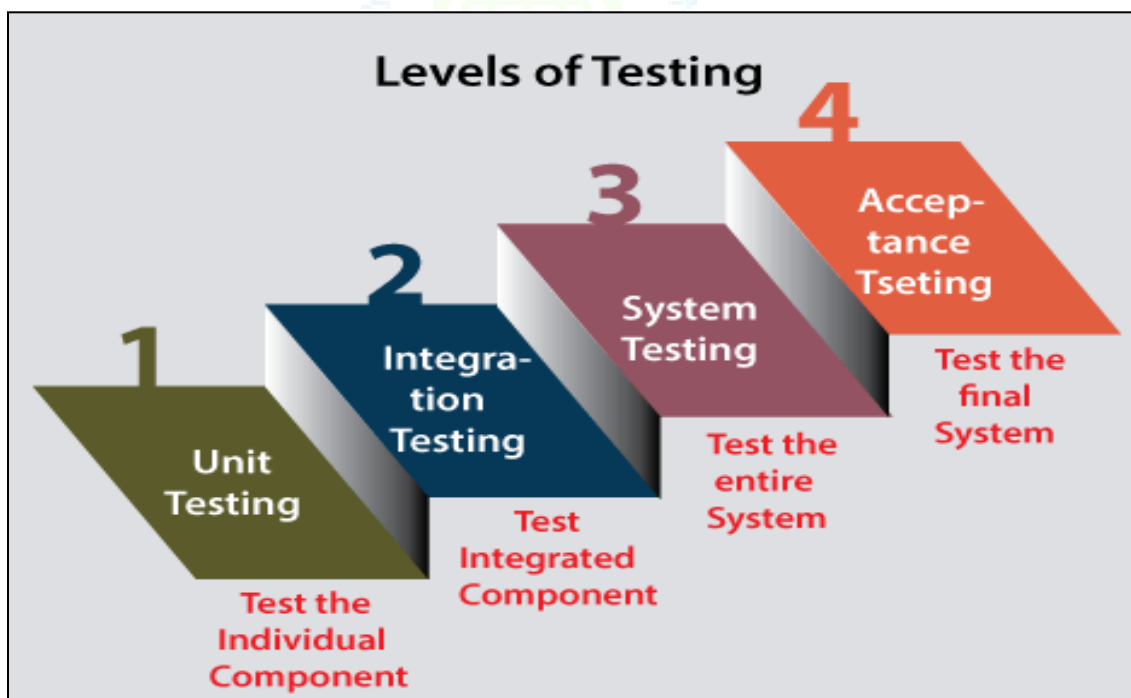
Step 6: Repeat for the next feature, like a "Forgot Password" option.

Benefits of Agile Testing and TDD:

1. **Faster Bug Detection:** Since testing is done after every small change, bugs are found early.
2. **Better Code Quality:** Code is written to pass specific tests, ensuring reliability.
3. **Customer Satisfaction:** Regular feedback from customers ensures the final product meets their needs.
4. **Reduced Development Time:** Continuous testing and quick fixes save time in the long run.
5. **Easier Maintenance:** Clean and tested code is easier to maintain and update.

Traditional Testing	Agile Testing
Testing is done after coding is complete.	Testing is done continuously during coding.
Follows rigid plans and documentation.	Flexible and adapts to changing requirements.
Testing is handled by a separate team.	Developers and testers work together.
Slow feedback due to long cycles.	Quick feedback after every sprint.

2.9. Levels of Testing



1. Unit Testing

It is the first level of testing. A unit refers to a single component or module of the software product that can be compiled, tested, and executed. We refer to unit testing as component testing as well.

The principal goal of unit testing is to confirm that each module of the software product is fulfilling its intended purpose. Generally, software developers are in charge of performing unit testing. It is a white box testing technique as developers test the code of every software

component.

Example: Testing a login function to ensure it correctly verifies user credentials.

Advantages:

- ✓ Identifies bugs early in development.
- ✓ Improves code quality by ensuring each module works correctly.
- ✓ Makes debugging easier as each unit is tested separately.

Disadvantages:

- ✗ Does not detect integration issues between modules.
- ✗ Requires detailed knowledge of code structure.
- ✗ Time-consuming to create unit test cases for every component.

2. Integration Testing

It is the second level of software testing

It is mainly used to test the data flow from one module or component to other modules.

Performing integration testing is the responsibility of software testers.

The primary purpose of executing the integration testing is to identify the defects at the interaction between integrated components or units.

When each component or module works separately, we need to check the data flow between the dependent modules, and this process is known as **integration testing**.

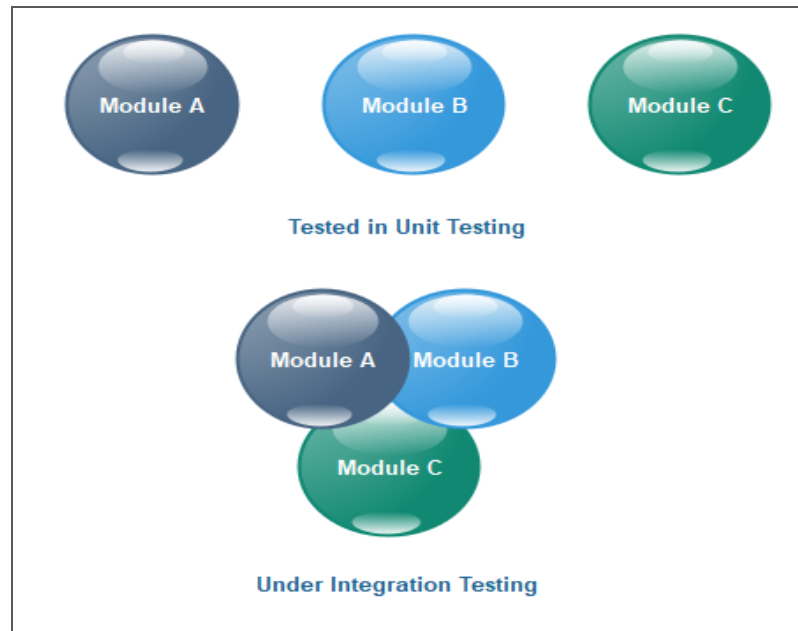
Example: Checking if the login system properly connects with the user database.

Advantages:

- ✓ Discloses issues in the interaction of the software components.
- ✓ Ensures that components after integration function properly.
- ✓ Bugs detected at this level are easier to fix.

Disadvantages:

- ✗ Can be complex in large systems.
- ✗ Requires proper test environment setup.

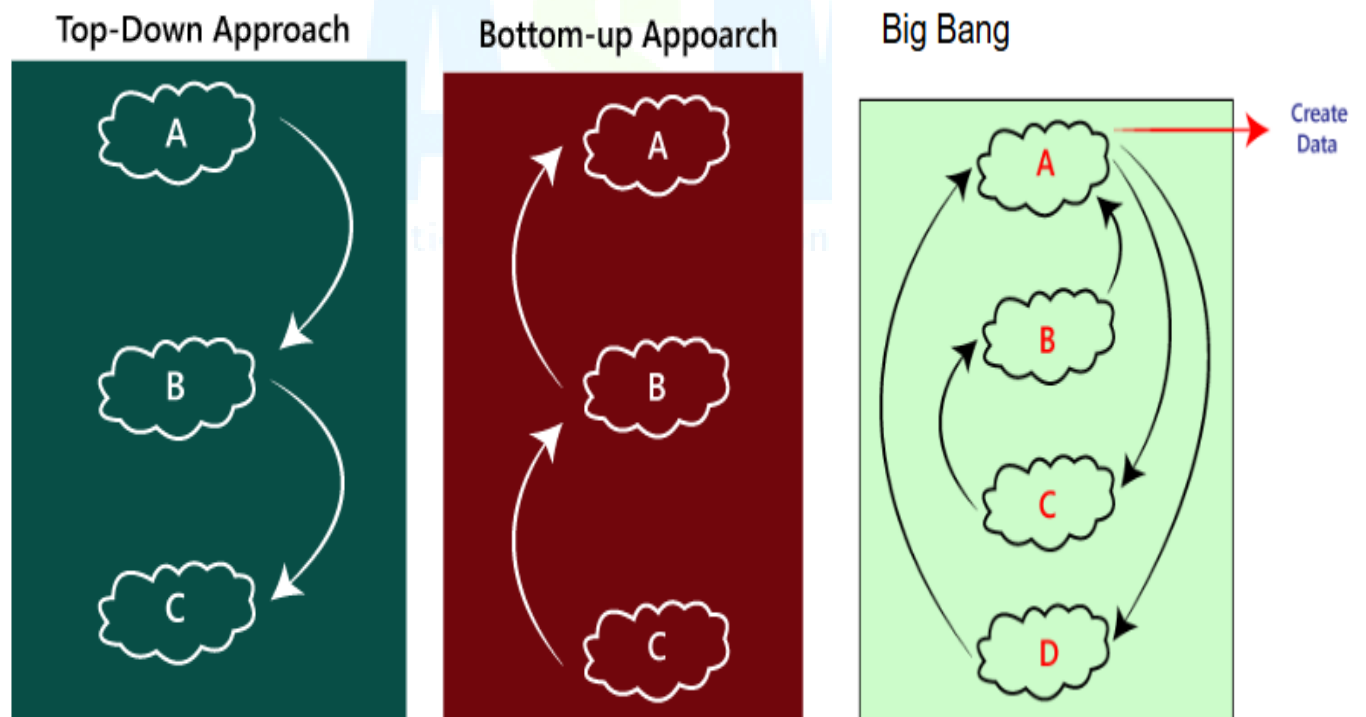


Top-Down Testing – Testing starts from the top module and proceeds downward.

Bottom-Up Testing – Testing starts from the lower modules and moves upward.

Big Bang Testing – All modules are tested together after development.

Incremental Testing – Modules are tested step by step in a controlled manner.



3. System Testing

- The third level of software testing is system testing, which is used to test the software's functional and non-functional requirements.
- In the third level of software testing, we will test the application as a whole system.
- To check the end-to-end flow of an application or the software as a user is known as System testing.
- Performing system testing is the responsibility of software testers.
- It is a black-box testing technique as it does not peer into the code or internal structure of the software product.

Types of System Testing:

- **Functional Testing** – Ensures the software meets functional requirements.
- **Performance Testing** – Checks speed, responsiveness, and stability.
- **Security Testing** – Identifies vulnerabilities in the system.
- **Usability Testing** – Evaluates user-friendliness and experience.

Advantages:

- ✓ Testers need not have programming knowledge to perform system testing.
- ✓ Evaluates the software product as a whole.
- ✓ Discloses errors that were left uncovered during the unit and integration testing.

Disadvantages:

- ✗ Time-consuming as compared to other testing levels.
- ✗ The cost required is pretty high as it entails testing the entire software product.

4. User Acceptance Testing(UTA)

The last level of software testing is acceptance testing. It involves the evaluation of the software product for its correctness by a small group of real users or clients and stakeholders. It is the final testing level carried out before making the software product live to customers.

UAT is the final level of testing where end users validate if the software meets business requirements and expectations.

Example: A banking application is tested by actual bank employees before launching for customers.

Types of UAT :

- ♦ **Alpha Testing** – Conducted by internal users in a controlled environment.
- ♦ **Beta Testing** – Conducted by real users in a live environment before full release.

Advantages:

- ✓ Ensures the product meets real-world needs.
- ✓ Reduces risks of failure after deployment.

Disadvantages:

- ✗ Requires coordination with end-users.
- ✗ Can be time-consuming.

2.10. Test Types

2.10.1. Functional testing (Black-box)

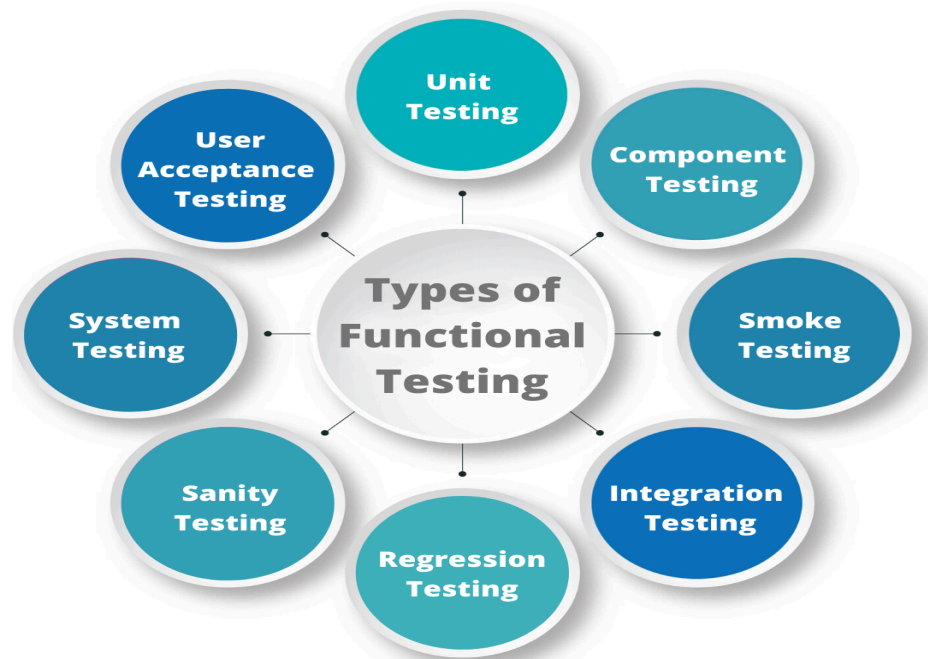
Functional testing is a type of software testing where the system is tested against the functional requirements or specifications. The primary objective is to verify that each function of the software application operates in conformance with the requirement specification. This testing method is also known as black-box testing because the internal structure or code of the application is not known to the tester.

Key Features of Functional Testing:

- Focuses on the external behavior of the software.
- Does not require knowledge of the internal code or logic.
- Tests are based on the software requirements and specifications.
- Involves inputting data and checking the output against the expected results.

Process of Functional Testing:

1. **Understand Requirements** : Analyze the functional requirements of the application.
2. **Test Case Design** : Create test cases and test data based on the requirements.
3. **Test Execution** : Run the test cases by providing the appropriate inputs.
4. **Output Verification** : Compare the actual output with the expected output.
5. **Defect Logging** : Report any deviations or defects found during testing.
6. **Retesting** : Verify the fixes and ensure that the functionality works as expected.



Advantages of Functional Testing:

- Ensures that the software meets user expectations.
- Detects missing functionality and interface errors.
- Easy to use and applicable to any level of testing.
- Does not require internal knowledge of the code.

Example of Functional Testing:

For a login page:

- **Input:** Username and Password.
- **Expected Output:** Successful login message if the credentials are correct.
- **Actual Output:** Verify if the system shows the correct message or error.

2.10.2. Non-functional testing (Testing of software product characteristics)

- **Non-functional testing** is a type of software testing that evaluates the non-functional aspects of a software application, such as performance, usability, reliability, and scalability.
- Unlike functional testing, which focuses on what the system does, non-functional testing emphasizes how well the system performs under various conditions.
- Non-functional testing is also very important as functional testing because it plays a crucial role in customer satisfaction.

- For example, non-functional testing would be to test how many people can work simultaneously on any software.

Key Characteristics of Non-Functional Testing:

- Evaluates the software's readiness based on quality attributes.
- Focuses on parameters such as speed, scalability, stability, and security.
- Performed after functional testing.
- Ensures that the application meets the non-functional requirements specified in the Software Requirement Specification (SRS).

Importance of Non-Functional Testing:

- Enhances software performance and efficiency.
- Improves user satisfaction by ensuring a seamless experience.
- Identifies potential bottlenecks and vulnerabilities.
- Ensures that the software is secure, scalable, and reliable.
- Helps in ensuring that the product meets quality standards and regulatory requirements.

Examples of Non-Functional Testing:

- **Performance Testing Example :** Testing how many users an e-commerce website can handle during peak sales.
- **Security Testing Example :** Checking for vulnerabilities in a banking application to prevent unauthorized access.
- **Compatibility Testing Example :** Ensuring that a mobile app works on both Android and iOS devices.

Education • Research • Consultancy

2.10.3. Structural testing (White-box)

- **Structural Testing**, also called **White-box Testing** or **Glass box testing**, is a type of software testing where the internal structure, code, and logic of the program are tested.
- The tester needs to have knowledge of the internal code to design test cases.
- The structural testing process requires an in-depth knowledge of the programming language and is opposite to Functional Testing.
- Imagine the software as a box. In white-box testing, you can see inside the box (the code), so you test the internal workings.

Key Features of Structural (White-box) Testing:

1. **Tests the internal code logic:** Checks how the code is written and if it works correctly.
2. **Covers paths and conditions:** Ensures every possible path in the code is tested, including loops and conditions (like if-else statements).
3. **Focus on code structure:** Helps find hidden errors, unreachable code, and logical mistakes.

Benefits of White-box Testing:

- Finds bugs early in the code.
- Helps improve code quality.
- It removes the dead code (extra code) or statements easily
- Ensures that all code paths are executed and tested.

Types of Structural Testing:

- **Statement Coverage:** Tests every line of code at least once.
- **Branch Coverage:** Tests every decision point (like **if-else** conditions).
- **Path Coverage:** Tests all possible paths in the program.

2.10.4. Testing related to changes – Confirmation (Re-testing) and Regression Testing

1. Confirmation Testing (Re-testing):

- Retesting is a process to check specific test cases that are found with bugs in the final execution.
- Generally, testers find these bugs while testing the software application and assign it to the developers to fix it. Then the developers fix the bug/s and assign it back to the testers for verification. This continuous process is called **Retesting**
- **Re-testing** is done to check if the bug is fixed correctly. It focuses only on the specific part where the bug was found, not the whole software.
- **Example:** Imagine an app has a bug where the login button doesn't work. After the developer fixes it, **Re-testing** ensures the login button now works properly.

2. Regression Testing:

- When changes are made to the software (like bug fixes, updates, or adding new features), **Regression Testing** checks if these changes have affected the existing functionalities.

- It tests the whole system to make sure nothing else is broken after the changes.
- After fixing the login button, **Regression Testing** will also check if other parts of the app (like the signup page, home screen, or profile section) still work fine.

Re-testing	Regression Testing
Testing the specific part of the software where a defect was fixed.	Testing the entire software to ensure that recent changes haven't broken existing functionality.
Focuses only on the specific bug or issue that was fixed.	Focuses on checking the overall system after any new code changes.
To confirm that a specific bug has been fixed correctly.	To ensure that new changes (fixes, updates) don't negatively affect the existing system.
Performed after a defect is fixed.	Performed after any code change, such as bug fixes, new features, or updates.
Narrow scope (tests only the fixed part).	Broader scope (tests many or all parts of the system).
If a login bug is fixed, only the login function is tested again.	After the login bug is fixed, the entire app is tested to ensure other features (like sign-up, profile, etc.) still work.
Re-testing is a planned test.	Regression testing is generic testing.
We do retesting only on failed test cases.	We do regression testing on passed test cases
Priority of retesting is higher than regression testing	Priority of regression testing is lower than retesting
Retesting is carried out before Regression testing.	Regression testing is carried out in parallel with retesting
We cannot automate the test cases for Retesting due to uncertainty	We can automate regression test cases

2.11. Non-Functional Testing Types

2.11.1. Performance (Load & Stress)

- **Performance Testing** checks how fast the software works under normal conditions.

Example: Testing if an e-commerce website loads quickly when 100 people are shopping at the same time.

- **Load Testing** checks how the software performs when many users are using it at once.

Example: Seeing if a ticket booking app works smoothly when many people try to buy concert tickets at the same time.

- **Stress Testing** checks how the software behaves under extreme conditions (more than usual usage).

Example: Adding thousands of users to an app at once to see when it crashes and if it can recover after crashing.

2.11.2. Usability

Focuses on the user interface and overall user experience.

Ensures that the application is easy to use and provides a satisfactory experience to end-users.

Example: Testing a mobile banking app to see if users can easily transfer money or check their balance without confusion.

2.11.3. Maintainability

Evaluates how easily the system can be maintained, including fixing bugs, updating code, and making modifications.

Example: Making sure the code of a shopping website is clean and well-documented so developers can quickly add new features or fix bugs.

2.11.4. Portability

Checks whether the software can operate on different hardware platforms, operating systems, browsers, or devices.

Ensures that the software is adaptable and works seamlessly across various environments without requiring significant changes.

Example: Testing a game to see if it runs smoothly on Windows, Mac, Android, and iOS without any issues.

2.11.5. Security

Focuses on identifying vulnerabilities, risks, and threats in the software application.

Ensures data integrity, confidentiality, and protection against unauthorized access, hacking, and other security breaches.

Example: Testing an online banking app to make sure that only the account holder can access their account and transactions are secure.

2.11.6. Localization & Internationalization

- **Localization Testing:** Checks if the software works well in different languages and regions.
 - Example: Testing a shopping app to see if it shows prices in rupees for India, dollars for the USA, and euros for Europe, along with correct language translations.
- **Internationalization Testing:** Checks if the software can easily support multiple languages and regions without code changes.
 - Example: Making sure an app's design and code can quickly adapt to different languages like English, Spanish, or Japanese when needed.

2.12. Concept of Smoke testing and Sanity Testing

Smoke Testing :

- Smoke testing is like a quick health check-up for the software. It is performed to ensure that the basic and critical features of the application are working fine before conducting detailed testing.
- Smoke Testing comes into the picture at the time of receiving build software from the development team.
- The purpose of smoke testing is to determine whether the build software is testable or not. It is done at the time of "building software."

- In smoke testing, we only perform positive testing, which implies that we can enter only the valid data not invalid data.
- Smoke testing is Subset of acceptance testing

Why is it called "Smoke Testing"?

The term comes from hardware testing, where engineers would power on a device for the first time and check if it starts up without producing smoke. Similarly, in software testing, if a build has major issues, it fails the smoke test and is rejected for further testing.

When is Smoke Testing Done?

- It is done **after a new software build** is developed.
- It helps testers quickly decide if the software is stable enough for deeper testing.

Example of Smoke Testing:

Imagine a team developing an online shopping website. After the first version of the website is created, the testers will check:

- Whether the homepage loads.
- Whether users can log in.
- Whether the product search function works.
- Whether the shopping cart adds items.

Sanity Testing

- Sanity testing is **a focused test done after making small changes** in the software to check if those changes work properly and do not affect other features.
- It is performed to check whether the bugs have been fixed after the build. Generally, Sanity testing is performed on stable builds.
- Sanity testing is a subset of regression testing.

Why is it called "Sanity Testing"?

The word "sanity" means "soundness of mind." This test ensures that the recent changes make sense and that the software is still behaving as expected.

When is Sanity Testing Done?

- It is done **after bug fixes or minor updates** to ensure that the specific issue is resolved.
- It checks only the affected functionality instead of testing the entire software.

Example of Sanity Testing:

Continuing with the shopping website example, assume users reported a bug where the "Add to Cart" button was not working. Developers fix the issue and send a new build.

The testers will now do **Sanity Testing** by checking:

- Whether the "Add to Cart" button works properly.
- Whether the selected items appear correctly in the cart.
- Whether other related features (like removing an item) still work.

